

Android开发中JSON的主流解析方式与选型建议

在Android开发中，JSON是最常用的数据交换格式之一，原生解析方式（如 `JSONObject` / `JSONArray`）适合简单场景，但实际开发中更推荐使用成熟的第三方库来提升效率、减少样板代码。下面重点讲解**原生解析**、**Gson**、**FastJson2**、**Moshi**这几种主流方式，涵盖核心用法和适用场景。

一、先明确JSON基础结构

JSON主要有两种核心结构：

- **对象**：`{}` 包裹，键值对形式（如 `{"name":"张三","age":20}`）
- **数组**：`[]` 包裹，元素列表（如 `[{"name":"张三"}, {"name":"李四"}]`）

二、1. 原生解析（无需依赖，适合简单场景）

Android内置了 `org.json` 包（`JSONObject` / `JSONArray`），无需引入第三方库，适合少量、简单的JSON解析。

核心API：

- `JSONObject`：解析JSON对象，通过 `getString()` / `getInt()` / `getBoolean()` 获取值
- `JSONArray`：解析JSON数组，通过 `getJSONObject(int index)` 获取数组中的对象

示例：

Code block

```
1  // 待解析的JSON字符串
2  String jsonStr = "{\"name\":\"张三\",\"age\":20,\"hobbies\":[\"篮球\",\"游戏\"]}";
3
4  try {
5      // 1. 解析JSON对象
6      JSONObject jsonObject = new JSONObject(jsonStr);
7      String name = jsonObject.getString("name"); // 张三
8      int age = jsonObject.getInt("age"); // 20
9
10     // 2. 解析JSON数组
11     JSONArray hobbiesArray = jsonObject.getJSONArray("hobbies");
12     for (int i = 0; i < hobbiesArray.length(); i++) {
```

```

13         String hobby = hobbiesArray.getString(i); // 篮球、游戏
14         Log.d("JSON", "爱好: " + hobby);
15     }
16 } catch (JSONException e) {
17     // 捕获解析异常（键不存在、类型不匹配等）
18     e.printStackTrace();
19 }

```

优缺点：

-  优点：无需依赖，轻量，适合简单场景（如单个对象/数组）
-  缺点：样板代码多、类型不安全（需手动强转）、嵌套JSON解析繁琐、容易抛 `JSONException`

2. Gson（Google出品，最主流）

Gson是Google开源的JSON解析库，支持**对象与JSON的双向转换**（序列化/反序列化），类型安全，适配绝大多数场景。

第一步：引入依赖

在 `app/build.gradle` （Module级别）添加依赖：

Code block

```

1  dependencies {
2      // Gson核心依赖
3      implementation 'com.google.code.gson:gson:2.10.1'
4  }

```

核心用法：

（1）基础：JSON字符串 → Java对象（反序列化）

先定义与JSON结构匹配的实体类（Bean/Model）：

Code block

```

1  // 实体类（字段名需与JSON键一致，或用@SerializedName指定别名）
2  public class User {
3      private String name;
4      private int age;
5      private List<String> hobbies;
6
7      // 必须有无参构造方法（Gson反射需要）

```

```

8     public User() {}
9
10    // getter/setter 可选 (Gson直接通过字段反射赋值)
11    public String getName() { return name; }
12    public void setName(String name) { this.name = name; }
13    // ... 其他getter/setter
14 }

```

解析代码：

Code block

```

1  String jsonStr = "{\"name\":\"张三\",\"age\":20,\"hobbies\":[\"篮球\",\"游戏\"]}";
2
3  // 1. 创建Gson实例
4  Gson gson = new Gson();
5
6  // 2. 解析为用户对象 (核心: fromJson)
7  User user = gson.fromJson(jsonStr, User.class);
8
9  // 直接使用对象属性
10 Log.d("Gson", "姓名: " + user.getName() + ", 年龄: " + user.getAge());

```

(2) Java对象 → JSON字符串 (序列化)

Code block

```

1  User user = new User();
2  user.setName("李四");
3  user.setAge(22);
4  user.setHobbies(Arrays.asList("读书", "跑步"));
5
6  // 核心: toJson
7  String jsonStr = gson.toJson(user);
8  Log.d("Gson", "序列化结果: " + jsonStr);
9  // 输出: {"name":"李四","age":22,"hobbies":["读书","跑步"]}

```

(3) 进阶：解析泛型 (如List<User>)

Gson解析泛型需要用 `TypeToken` 指定类型：

Code block

```

1  String jsonArrayStr = "[{\"name\":\"张三\",\"age\":20},{\"name\":\"李四\", \"age\":22}]";

```

```

2
3 // 解析List<User>
4 List<User> userList = gson.fromJson(jsonArrayStr, new TypeToken<List<User>>()
    {}.getType());
5
6 // 遍历列表
7 for (User u : userList) {
8     Log.d("Gson", "用户: " + u.getName());
9 }

```

(4) 别名适配 (JSON键与字段名不一致)

用 `@SerializedName` 注解指定JSON中的键名：

Code block

```

1 public class User {
2     // JSON中键是"user_name", 字段名是name
3     @SerializedName("user_name")
4     private String name;
5     private int age;
6 }

```

优缺点：

- ✅ 优点：功能全面、稳定、易上手、支持泛型/别名/空值处理
- ❌ 缺点：性能略低于FastJson2，反射实现（但日常开发无感知）

3. FastJson2 (阿里出品，高性能)

FastJson2是FastJson的重构版，性能大幅提升，支持Java/Kotlin，适配Android全版本。

第一步：引入依赖

Code block

```

1 dependencies {
2     // FastJson2核心依赖
3     implementation 'com.alibaba.fastjson2:fastjson2:2.0.48'
4 }

```

核心用法：

(1) JSON → Java对象

Code block

```
1 String jsonStr = "{\"name\":\"张三\",\"age\":20}";
2
3 // 核心: JSON.parseObject
4 User user = JSON.parseObject(jsonStr, User.class);
```

(2) Java对象 → JSON

Code block

```
1 User user = new User();
2 user.setName("李四");
3 user.setAge(22);
4
5 // 核心: JSON.toJSONString
6 String jsonStr = JSON.toJSONString(user);
```

(3) 解析JSON数组 (List<User>)

Code block

```
1 String jsonArrayStr = "[{\"name\":\"张三\",\"age\":20},{\"name\":\"李四\", \"age\":22}]";
2
3 // 解析List<User>
4 List<User> userList = JSON.parseArray(jsonArrayStr, User.class);
```

(4) 泛型解析 (复杂场景)

Code block

```
1 // 解析Map<String, User>
2 String jsonMapStr = "{\"user1\":{\"name\":\"张三\",\"age\":20}}";
3 Map<String, User> userMap = JSON.parseObject(jsonMapStr, new
    TypeReference<Map<String, User>>(){});
```

优缺点:

-  优点: 性能顶级 (比Gson快)、API简洁、支持多格式 (JSONB/UTF8)
-  缺点: 历史版本 (FastJson1) 有安全漏洞, 但FastJson2已修复; 部分场景兼容性略逊于Gson

4. Moshi (Square出品, Kotlin友好)

Moshi是Square（Retrofit/OkHttp开发者）推出的解析库，专为Kotlin优化，支持协程，适合Kotlin项目。

第一步：引入依赖

Code block

```
1 dependencies {
2     // Moshi核心
3     implementation 'com.squareup.moshi:moshi:1.15.0'
4     // Kotlin适配（如果用Kotlin）
5     kapt 'com.squareup.moshi:moshi-kotlin-codegen:1.15.0'
6 }
```

核心用法（Kotlin示例）：

Code block

```
1 // 1. 定义数据类
2 data class User(
3     val name: String,
4     val age: Int,
5     val hobbies: List<String>
6 )
7
8 // 2. 创建Moshi实例
9 val moshi = Moshi.Builder().build()
10
11 // 3. 创建解析器
12 val userAdapter = moshi.adapter<User>(User::class.java)
13
14 // 4. JSON → User
15 val jsonStr = "{\"name\":\"张三\",\"age\":20,\"hobbies\":[\"篮球\",\"游戏\"]}"
16 val user = userAdapter.fromJson(jsonStr)
17
18 // 5. User → JSON
19 val json = userAdapter.toJson(user)
```

优缺点：

-  优点：Kotlin友好、无反射（代码生成）、与Retrofit/OkHttp生态适配好
-  缺点：Java项目使用略繁琐，生态比Gson小

三、选型建议

场景	推荐库
Java项目、简单/通用	Gson
高性能要求、大数据	FastJson2
Kotlin项目、协程	Moshi
极小体积、无依赖	原生JSONObject

四、通用注意事项

- 1. 实体类字段类型要与JSON一致（如JSON中是数字，字段不能用String）；
- 2. 避免空指针：解析前判空，或用库的空值处理配置；
- 3. 网络JSON解析建议在子线程执行（避免主线程阻塞）；
- 4. 敏感数据（如密码）解析后及时清理，避免内存泄漏。

（注：文档部分内容可能由 AI 生成）